

PYTHON

Dr. **Bülent Çobanoğlu**

cobanoglubulent[at]gmail.com

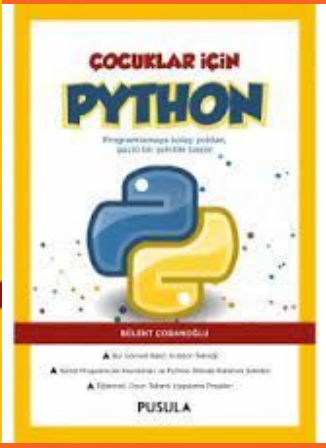
Ben kimim?

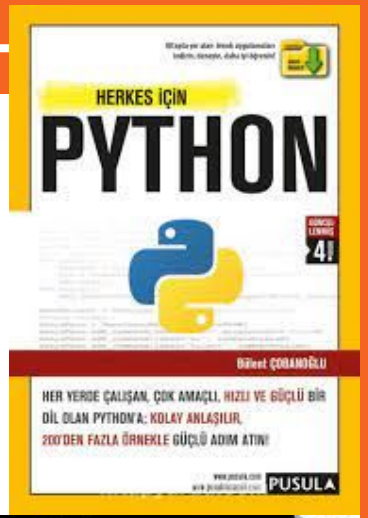
Kaynaklar;

- Eğitmen notları, https://github.com/bcobanoglu
- Python resmi sitesi

https://docs.python.org/3/









Problem çözme becerisini artırmaya yönelik adresler;

- https://codescracker.com/exam/showtest.php?subid=10
- https://realpython.com/search?kind=quiz&q=quizs
- https://www.tutorialspoint.com/python/python_questions_answers.htm
- https://pynative.com/
- https://www.w3resource.com/python/python-tutorial.php
- https://www.codewars.com/collections/basic-python
- https://www.hackerrank.com/domains/python
- https://edabit.com/challenges/python3
- https://rosettacode.org/wiki/Category:Programming_Tasks
- https://projecteuler.net/archives

BootCamp İçeriği

- **Module 1**

. Python’a giriş ve Çalışma ortamını hazırlama

- Veri bilimi için Niçin Python?
- Python kod derleme aşamaları
- Terminalden çalışmak
- Jupyter ve Colab Notebook ile çalışma
- print, input, help, dir kullanımı
- Script-Modül, Paket kavramı ve paket indirme
- .py, .pyc, .ipynb dosyaları ve farkları

. Comments, Docstrings

. PEP

. Değişken, Sabit tanımlama

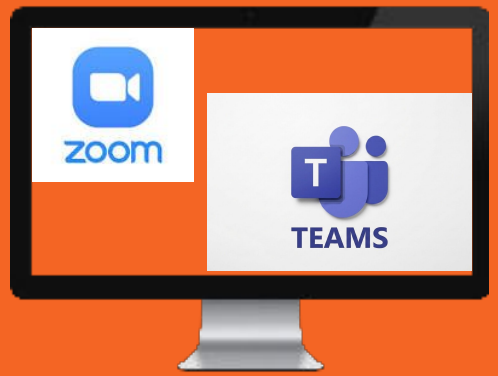
İçindekiler

- **Module 2:** Python temel programlama kavramları ve String işlemler
  - Veri tipleri ve Dönüşümü
  - Operatorler
  - Sayı sistemleri ve dönüşümleri
  - String formatlama
  - String fonksiyonlar
  - Karar yapıları ve döngüler
  - Fonksiyonlar
  - Argüman ve parametre kavramları
  - Hata yakalama ve yönetimi
- **Module 3: Koleksiyonlar ve Veri yapıları**
  - **List, Tuple, Set, Dictionary**
  - **Listelerden Stack ve Queue veri yapılarını oluşturma**
  - **Unpacking, zip, map, filter, reduce**

İçindekiler

- **Module 4: Dosya ve Veritabanları ile çalışmak**
  - Metin dosyalarını okuma/yazma işlemleri
  - Excel, csv verileri ile çalışmak
  - json verileri ile çalışmak
- **Module 5: Pandas ile çalışmak**
  - **Numpy ve Pandas kütüphanesi**
  - **Pandas kütüphanesi ile Series ve Data Frame kullanımı**
  - **Pandas kütüphanesi ile Dosya İşlemleri**
  - **Pandas ile kayıp veri analizi**
  - **Pandas ile Dataframe yapılarının birleştirmesi, düzenlenmesi**
- **Module 6: Veri Görselleştirme**
  - **Matplotlib ile veri görselleştirme**
  - **Matplotlib ile animasyon oluşturma**
  - **Seaborn ile görselleri zenginleştirme**

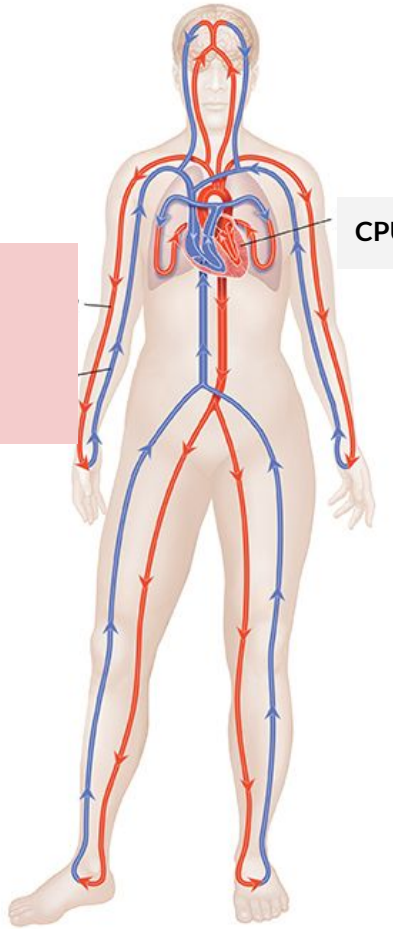
Araçlar



- COLAB/Jupyter NOTEBOOK
- peardeck
- Quiz(Kahoot,...)
- ....

Programlama Dilleri ve PYTHON

Python
ve diğer
programlama
dilleri
(yazımlar)



Programlama Nasıl Öğrenilir?

> “Programming is learned by writing programs.”
>
> —Brian Kernighan

1- Problemi (öncelikle) kendin çözmeye çalış;

2- Kodu notebooka (kağıda) **yaz;**

3- Notebook’a yazdığımız kodu test et;

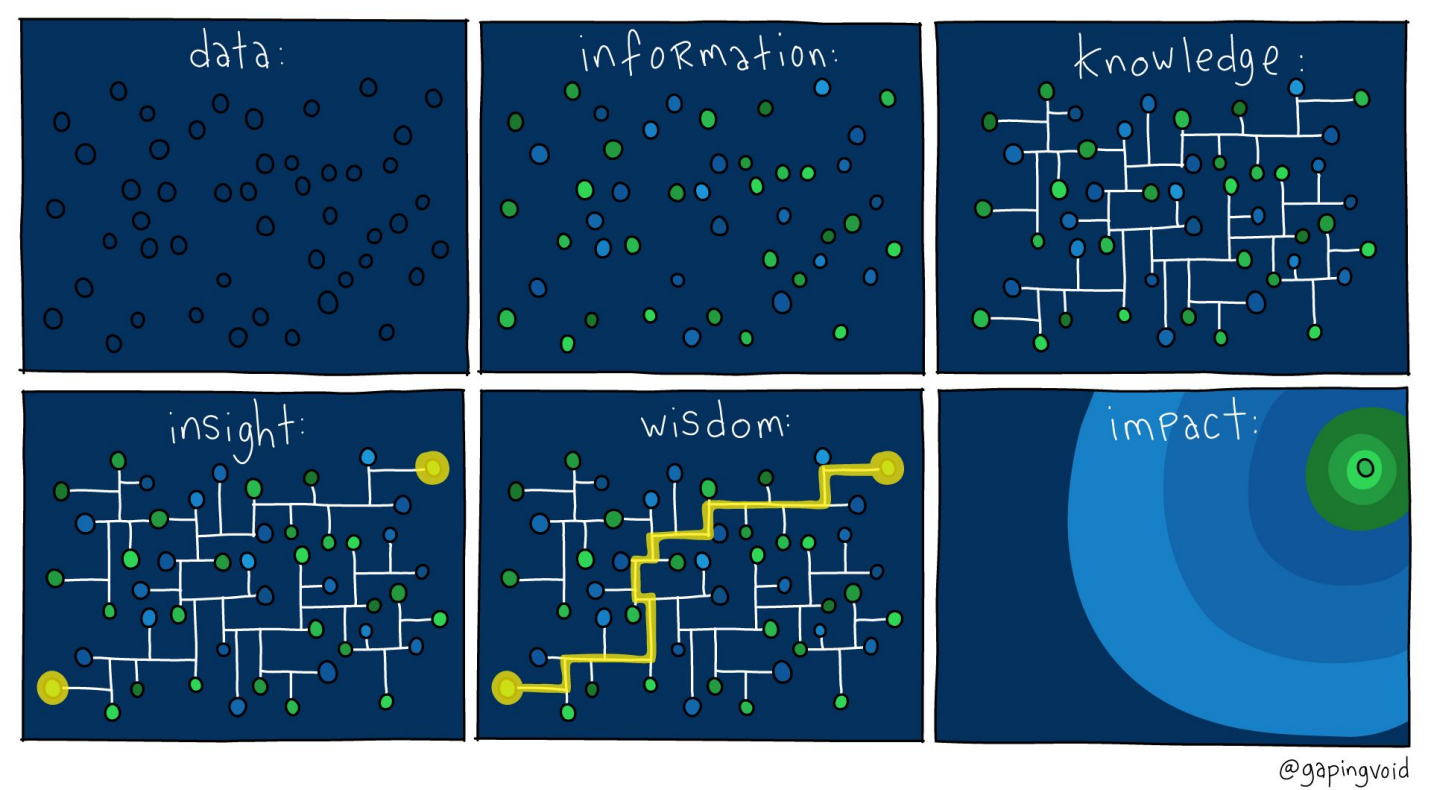
4- Olası hataları AI desteği ile gider ve nasıl düzelttiğini not et;

Q: Bütün yazılımlar bir programlama dili ile yazıldığına göre programlama dilleri nasıl yazılmıştır?



Data ve evrimsel yolculuğu?

Data is the new oil. It needs to be refined.

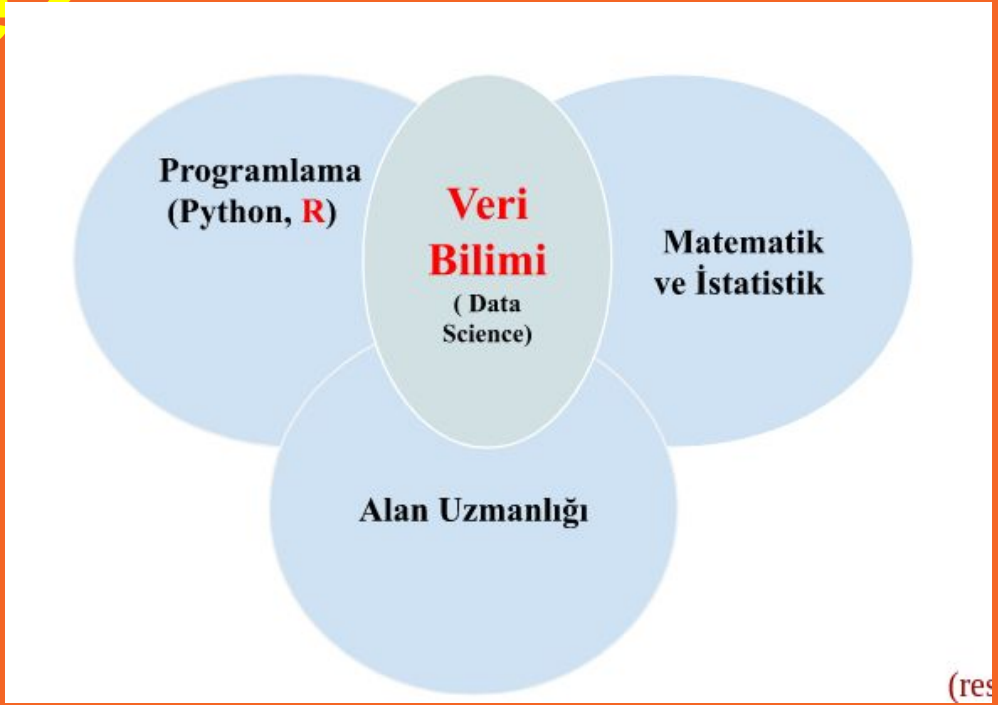


15112025 ->15.11.2025 -> 15.11.2025 -> -500.000->?

Data ve evrimsel yolculuğu

İngilizce Terim	Türkçe Karşılığı	Açıklaması	Cevapladığı Soru
Data	Veri	Ham, işlenmemiş gerçekler, sayılar, semboller, gözlemler.	-
Information	Enformasyon/Bilgi	İşlenmiş, düzenlenmiş, anlamlandırılmış veri.	Ne? Ne zaman? Nerede?
Knowledge	Bilgi/Malumat	Anlaşılmış, yorumlanmış, deneyimle birleştirilmiş enformasyon.	Nasıl?
Insight	İçgörü	Derinlemesine analiz sonucu elde edilen yeni ve anlamlı anlayış, örüntülerin keşfi.	Neden?
Wisdom	Bilgelik/Hikmet	İçgörülerin deneyim, etik değerler ve uzun vadeli perspektifle birleşimi, doğru karar verme yeteneği.	Öyleyse ne?
Impact	Etki/Tesir	Bilgelik temelinde alınan kararların ve uygulanan eylemlerin yarattığı sonuçlar.	-

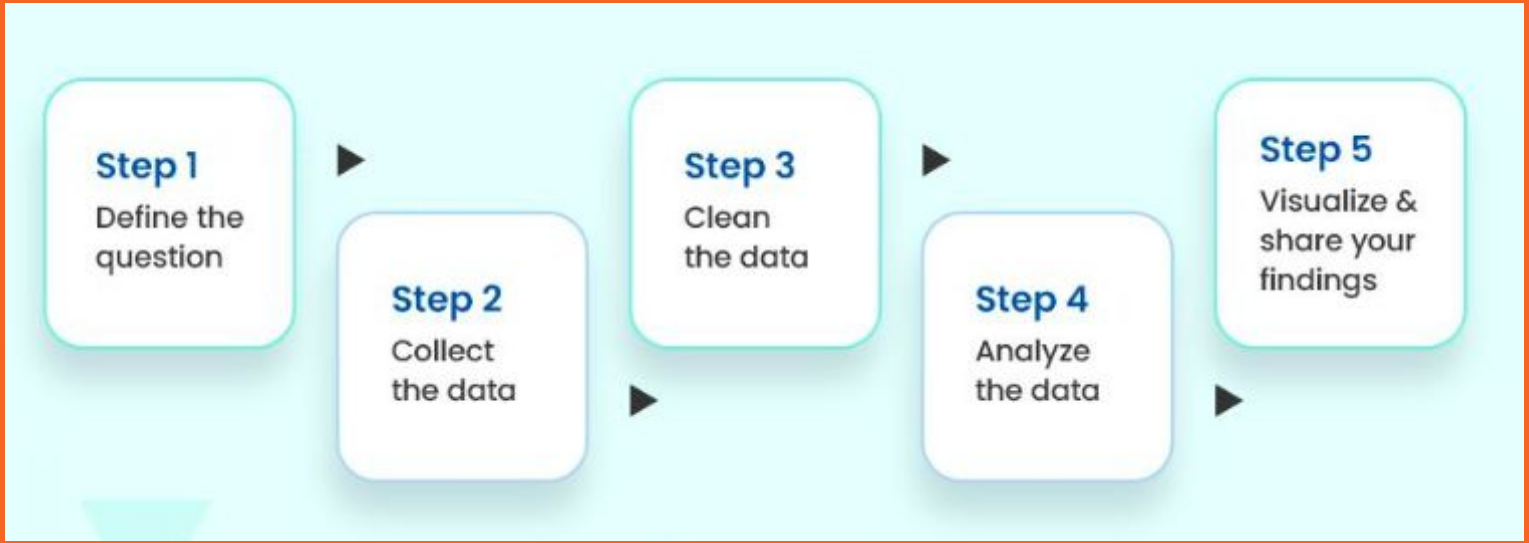
Data Science?



Veri bilimi (data science); *verilerden bir değer elde etmek ve anlamlı çıkarımlar yapabilmek için* birçok alanı bir araya getiren **çok disiplinli bir bilim dalıdır.**

Veri Anal

- Tipik bir veri bilimi projesinin aşamaları neye benzer?
- *(%80 şuna benzer):*



Problem Tanımı -> Veri setini al/Topla -> Düzenle ve Temizle -> Analiz et ve Dönüştür-> Görselleştir(Visualise) -> Sonuçları Paylaş

Q: Veri bilimi için tercih edilen diller neler? Avantaj ve dezavantajları hakkında neler söylersiniz?



Big Data?



5V

1. **Volume (Hacim)**: Terabaytlardan petabaytlara kadar büyük miktarda veri
2. **Velocity (Hız)**: Verinin üretilme ve işlenme hızı yüksektir.
3. **Variety (Çeşitlilik)**: Yapılandırılmış(*SQL*), yarı yapılandırılmış (*JSON*) ve yapılandırılmamış(*video, resim*) veri türleri

Sonradan eklenen diğer özellikler:

- **Veracity** (Doğruluk) - Verinin kalitesi ve güvenilirliği. Büyük veri kümelerinde tutarsızlıklar, eksiklikler ve gürültülü veriler olabilir.
- **Value** (Değer) - Büyük verinin analiz edilerek anlamlı bilgiler ve içgörüler elde edilmesi

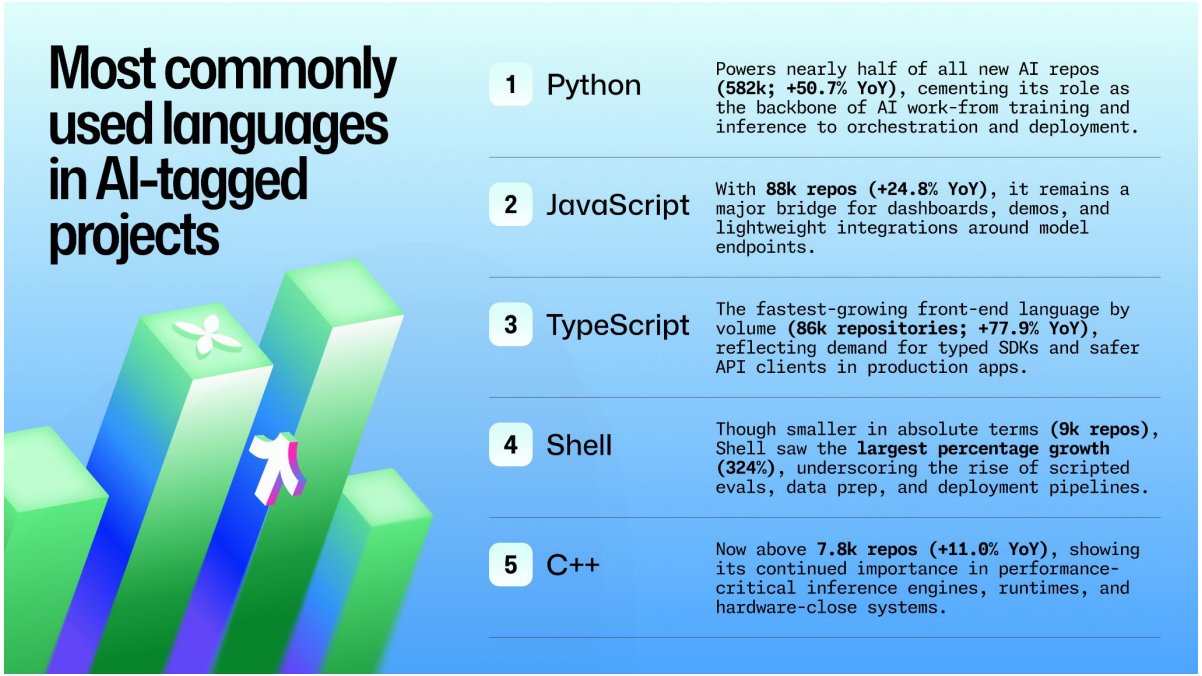
Niçin PYTHON?

Programlama Dili	2026	2021	2016	2011	2006	2001	1996	1991	1986
Python	1	3	5	6	8	25	24	-	-
C	2	1	2	2	2	1	1	1	1
C++	3	4	3	3	3	2	2	2	6
Java	4	2	1	1	1	3	15	-	-
C#	5	5	4	5	7	13	-	-	-
JavaScript	6	7	7	10	9	9	20	-	-
Visual Basic	7	6	13	-	-	-	-	-	-
SQL	8	9	-	-	-	37	-	-	-
R	9	11	17	27	-	-	-	-	-
Delphi/Object Pascal	10	19	12	12	10	36	-	-	-

https://www.tiobe.com/tiobe-index/

Niçin PYTHON?

- Python, 2,6 milyon katılımcısıyla (+%48 yıllık artış) **yapay zeka ve veri biliminde baskın** konumunu koruyor. Jupyter ve **Colab Notebook** ise yapay zeka için en çok tercih edilen ortam olmaya devam ediyor (yaklaşık 403 bin depo; yapay zeka etiketli projelerde +%17,8 yıllık artış).



https://github.blog/news-insights/octoverse/octoverse-2024/#the-most-popular-programming-languages

Niçin PYTHON?

Programlama, betik ve işaretleme dilleri

On yılı aşkın süredir istikrarlı bir büyüme gösteren Python'ın benimsenmesi önemli ölçüde hızlandı. 2024'ten 2025'e kadar %7'lik bir artış gösterdi; bu da yapay zeka, veri bilimi ve arka uç geliştirme için tercih edilen dil olma yeteneğini ortaya koyuyor.

Son bir yılda hangi **programlama, betik ve işaretleme dillerinde** yoğun geliştirme çalışmaları yaptınız ve önümüzdeki yıl hangi dillerde çalışmak istiyorsunuz? (Hem o dilde çalıştıysanız hem de çalışmaya devam etmek istiyorsanız, lütfen o satırdaki her iki kutuyu da işaretleyin.)



https://survey.stackoverflow.co/2025/technology#admired-and-desired

PYTHON Nasıl bir DİL?

- **Kolay** öğrenilen,
- **Kolay** kod yazılan,
- **Kolay** çalıştırılan,
- **Kolay** derlenen,
- **Kolay** program geliştirilen,
- **Kolay** öğretilen 😀 😀 😀



https://www.python.org/about/

PYTHON Nasıl bir DİL?



- **Güçlü,**
- **Hızlı,**
- **Kullanıcı dostu** (friendly)**,**
- **Açık kaynaklı**(Open source)
- **Yorumlayıcı tabanlı** (interpreted),
- **Her yerde çalışabilen** (platform independent)
- **Yüksek seviyeli** (High level)**,**
- **Genel amaçlı** (general-purpose) bir dil 😀

Python Tarihçe



@gvanrossum

Python Dili Geliştiricisi ve İsim Kaynağı

- 1991 yılında, Hollandalı bilgisayar mühendisi Guido van Rossum tarafından geliştirilmeye başlanmıştır.
- Python ismini, Monty Python adlı altı kişilik bir İngiliz komedi grubunun *Monty Python's Flying Circus* adlı gösterisinden almıştır.
- En güncel Python sürümünü www.python.org adresinden indirebilir, tüm dokümanlara bu adresten ulaşabilirsiniz.

Python Tarihçe



Q: Python Logosu ne zaman ve kim tarafından geliştirilmiştir?

python™

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Python Tarihçe

Python logo, used between 1990s–2006 ((Guido))



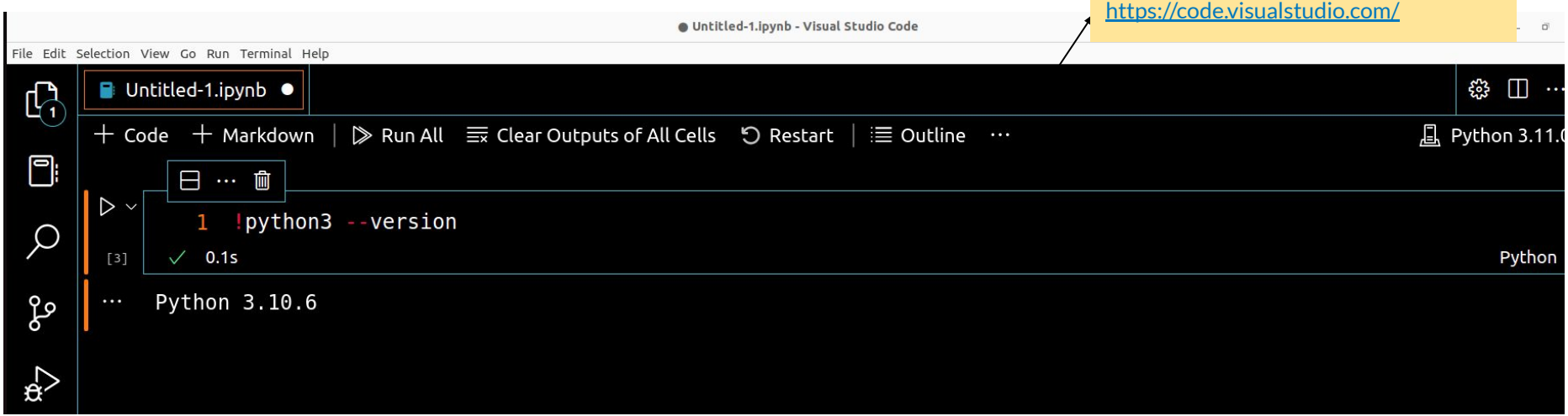
Python logo, used between 2006s–Today (Tim Parkin)



https://wiki.python.org/psf/Python%20Logos

KOD GELİŞTİRME ORTAMLARI (IDEs)

Visual Studio Code
https://code.visualstudio.com/



Google Colab Notebook
https://colab.research.google.com/



Python Editörleri

https://realpython.com/python-ides-code-editors-guide/

- **IDLE** *(Win. kurulumu ile gelen)*
- ***Thonny,*** ***(https://thonny.org/)***
- ***Geany,***
- ***Eric,***
- **Eclipse *PyDev,***
- ***PyCharm,***
- ***Visual Studio Code***

- Python Anywhere: https://www.pythonanywhere.com/
- **https://repl.it/languages/python3**
- ***https://www.jdoodle.com/python3-programming-online/***
- ***https://www.programiz.com/python-programming/online-compiler/***
- ***https://pythontutor.com/***

Data Science Uygulamaları için;

- ***Google Colab:*** ***https://colab.research.google.com/***
- ***Jupyter Notebook :*** ***https://jupyter.org/***
- `Spyder`
- `marimo`

Q: Bir Python sürümünün ömrü ortalama kaç yıldır?

https://devguide.python.org/versions/



Branch	Schedule	Status	First release	End of life	Release manager
main	PEP 826	feature	*2027-10-06*	*2032-10*	Savannah Ostrowski
3.15	PEP 790	prerelease	*2026-10-01*	*2031-10*	Hugo van Kemenade
3.14	PEP 745	bugfix	2025-10-07	*2030-10*	Hugo van Kemenade
3.13	PEP 719	bugfix	2024-10-07	*2029-10*	Thomas Wouters
3.12	PEP 693	security	2023-10-02	*2028-10*	Thomas Wouters
3.11	PEP 664	security	2022-10-24	*2027-10*	Pablo Galindo Salgado
3.10	PEP 619	security	2021-10-04	*2026-10*	Pablo Galindo Salgado

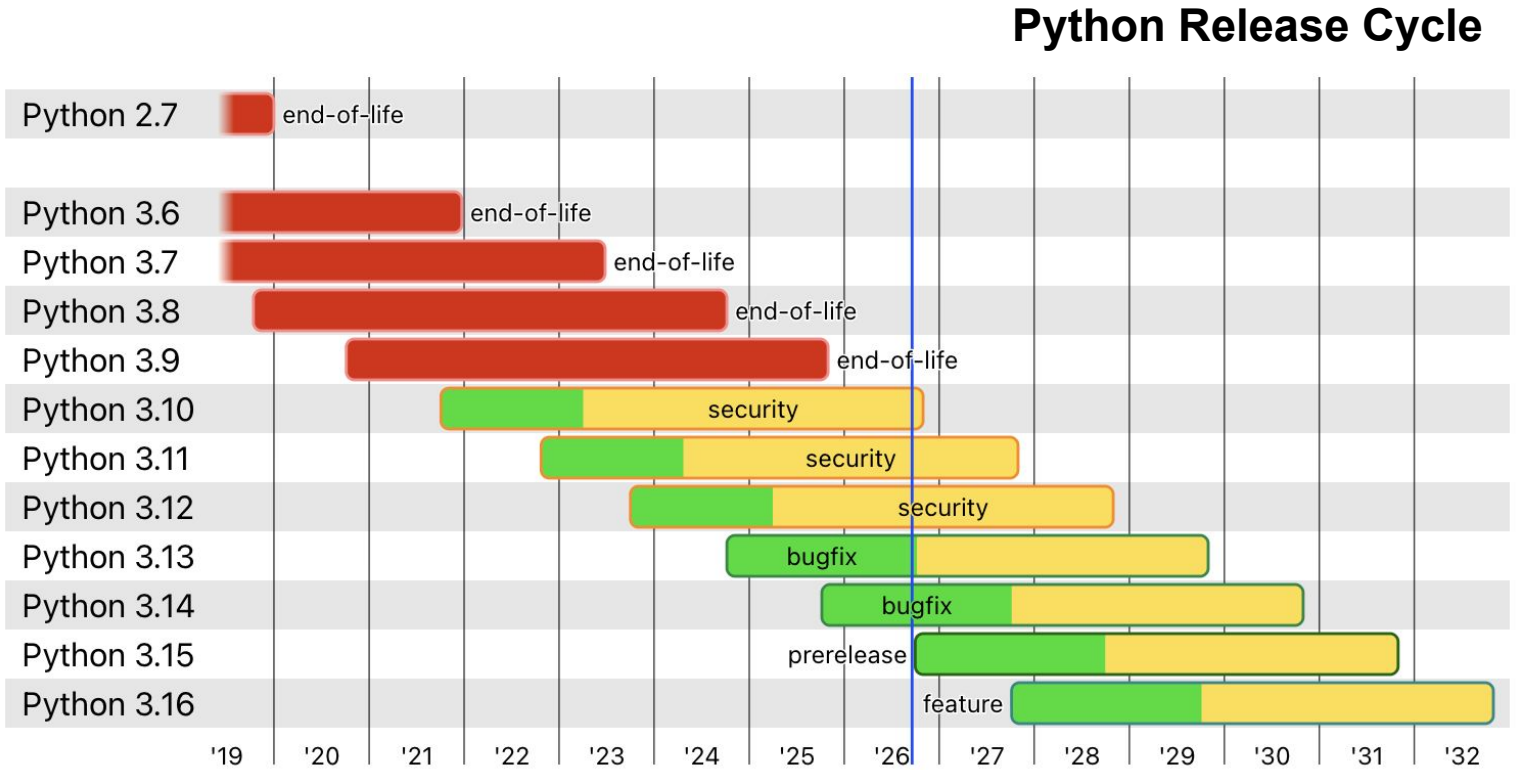


Students choose an option

Pear Deck Interactive Slide
Do not remove this bar

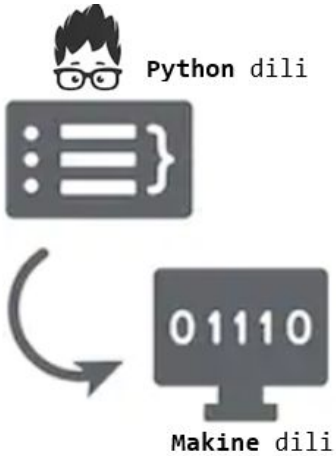
Python Sürümleri

https://devguide.python.org/versions/



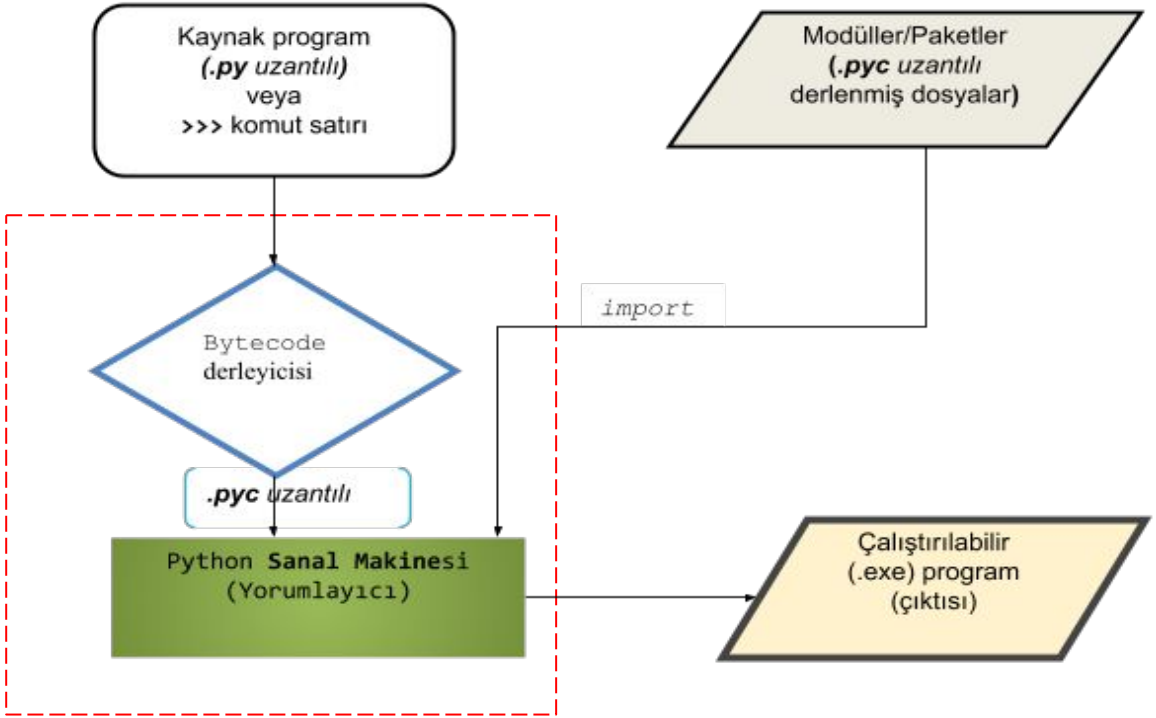
Python kodu ile Makine (PC) arasındaki iletişim nasıl gerçekleşir?

Python programlama dilinde yazılan bir dosyanın uzantısı '**.py**' dir.

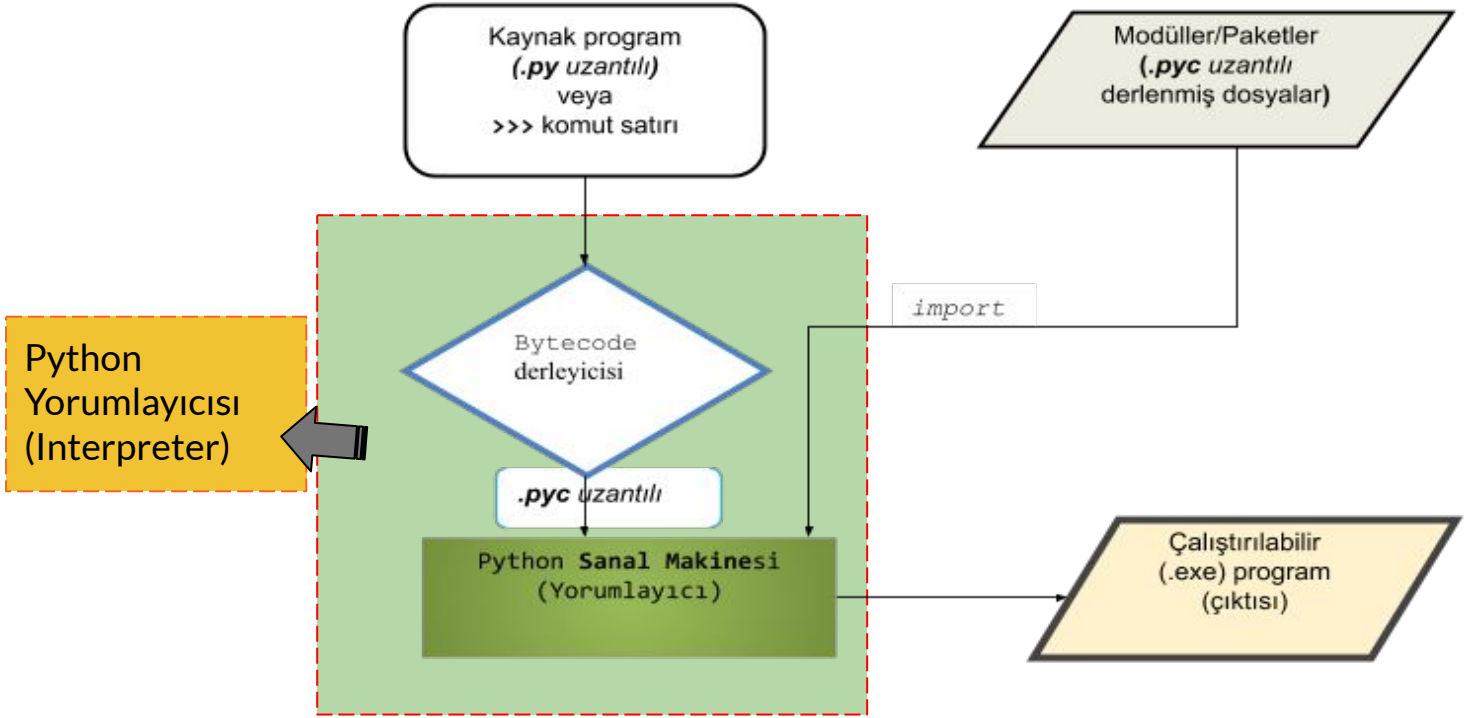


Programlama dilleri; kaynak kodun *(.py uzantılı dosyadan)* makine koduna *(.exe uzantılı dosyaya*) çevrilmesi ve çalıştırılması işleminde **derleyici, yorumlayıcı** veya **her ikisini birlikte** kullanabilirler.

Python kod derleme aşamaları



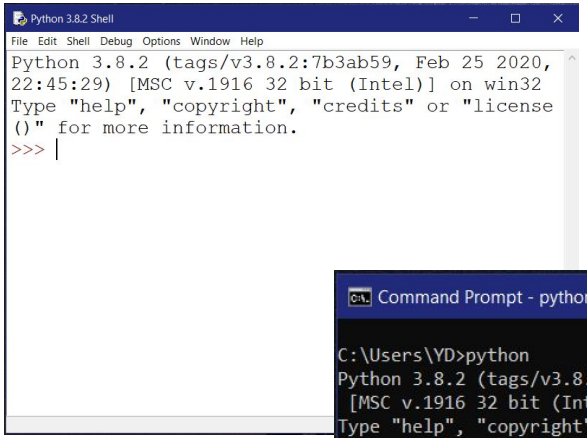
Python kod derleme aşamaları



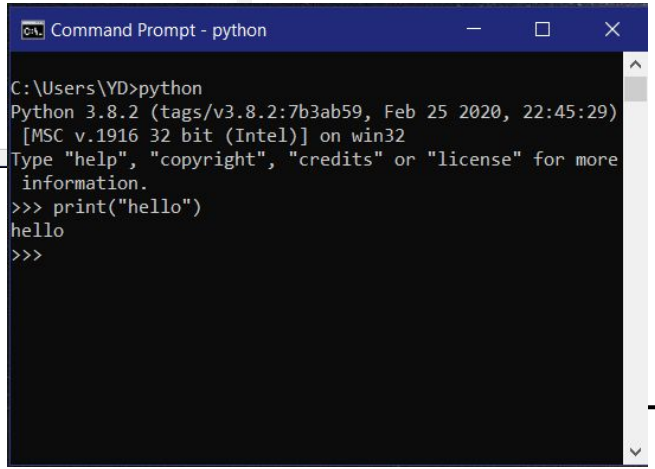
Bytecode (Bayt kod) derleyicisi, kaynak kodu (Python komutlarını) girdi olarak kabul eder. Ardından çıktı olarak makine tarafından okunabilecek Python bayt kodunu (.pyc uzantılı dosya) üretir. Sanal makine ise girdi olarak Python bayt kodunu (.pyc uzantılı) ve varsa ilgili kütüphane modüllerini alır, bytecode ile temsil edilen sanal makine talimatlarını çalıştırır, yani çalıştırılabilir (.exe) dosya formatına dönüştürür.

Python Kodu Nasıl Çalıştırılır?

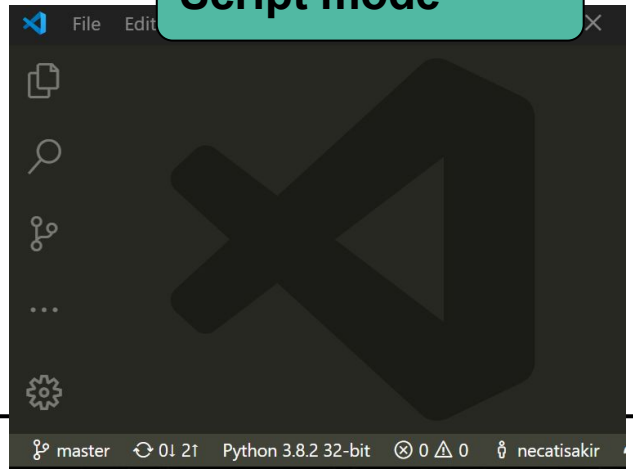
Interactive mode







Script mode



Q : ***.py*** ***ve .pyc*** **dosyaları arasındaki fark nedir?**



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Özellik	.py	.pyc
İçerik	Kaynak kod (metin)	Bytecode (ikili)
Kim oluşturur	Siz	Python, otomatik
Okunabilir mi	Evet	Hayır
Nerede durur	Proje klasörü	`__pycache__/`
Sürüme bağlı mı	Hayır	Evet
Paylaşılır mı	Evet	Genelde paylaşılmaz (Git'e eklenmez)

Not. script (`dosya.py`) dosyalar için .pyc oluşmaz, yalnızca import edilen modül dosyalar için oluşur.



Compiler(Derleyici) & Interpreter (Yorumlayıcı) Farkı?

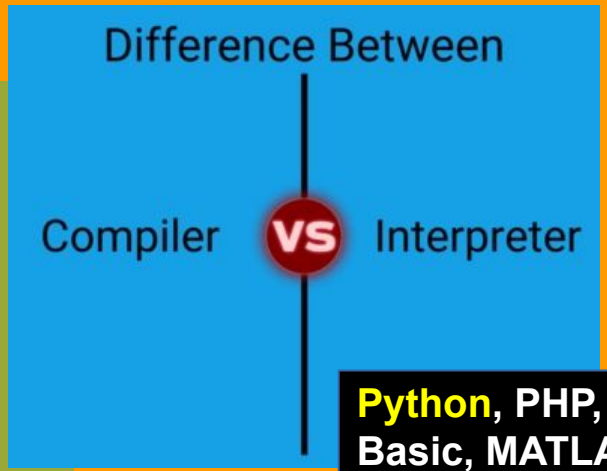
Q : Python hangisini kullanır?

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Derleyici – Yorumlayıcı Farkı?

- Pascal, C, C++, C#, dilleri derleyici tabanlı (compiler-based).
- Compiler, kaynak kodu amaç (object) koda dönüştürürler.
  - Tüm kodu birden çevirerek, Standart bir amaç kod üretirler.
  - Derleyici tün kaynak kodu derler Ve varsa tüm hataların bir listesini verir.

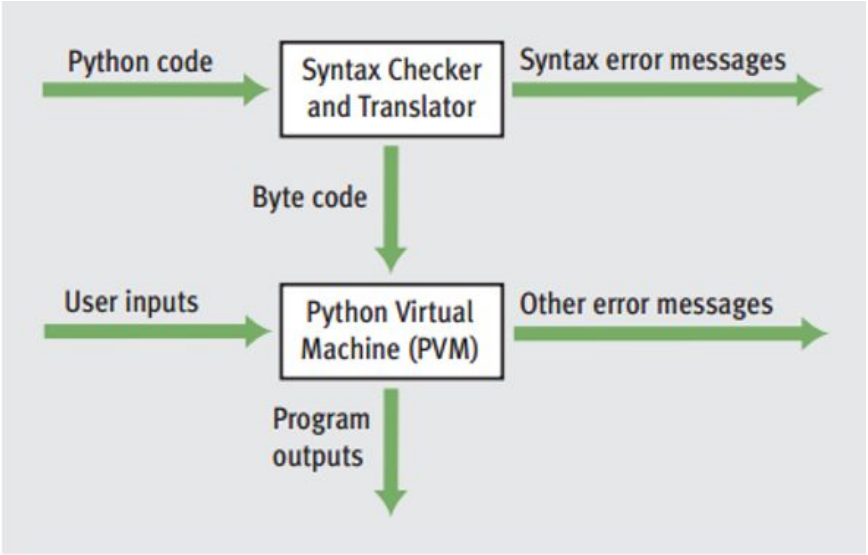


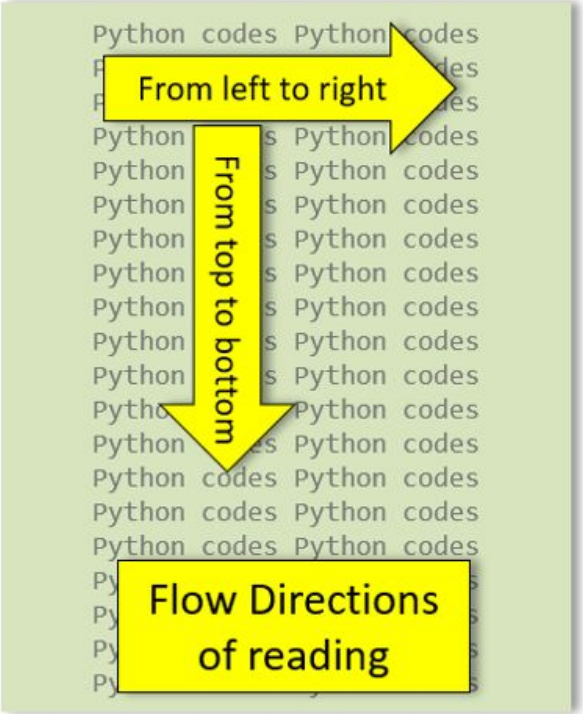
Python, PHP, Ruby, Perl, JavaScript, Basic, MATLAB dilleri yorumlayıcı tabanlı (interpreter-based).

-Yorumlayıcılar, kaynak kodu satır satır doğrudan çalıştırırlar.

- Yorumlayıcı ilk hatalı satırda durur.

Yorumlayıcı Ne Yapar?





Basitçe kodu okur; yorumlar.

Python Yorumlayıcıları

CPython Default(varsayılan) Python yorumlayıcısı dır. **C dilinde yazılmıştır.** En son dil özelliklerini, en yüksek Python paketi uyumluluğunu, tam hata ayıklama desteğini ve IPython ile birlikte çalışma özelliklerini içerir.

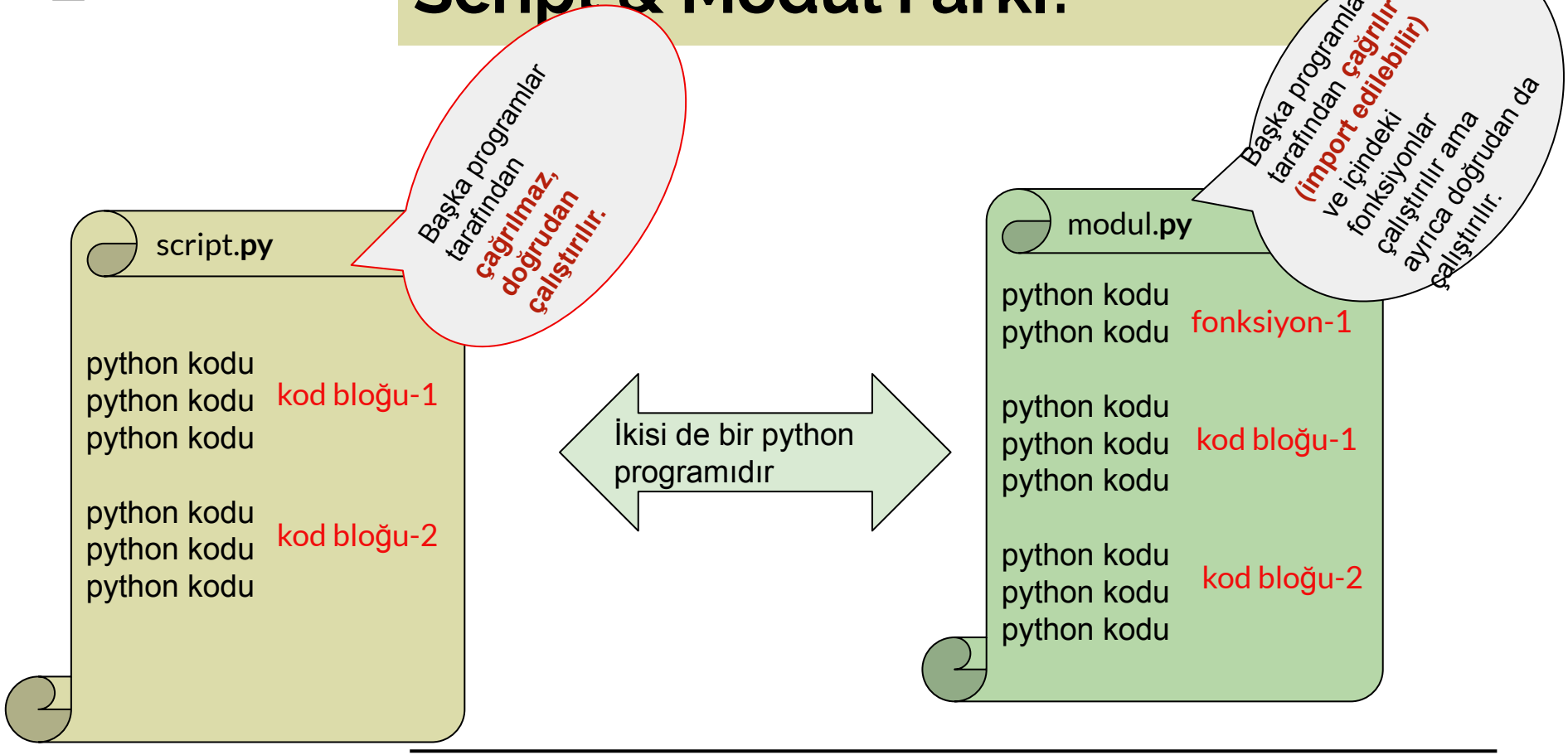
IronPython :NET tabanlı (C# dili ile yazılmış) Python yorumlayıcısıdır. IronPython, sanal ortamları desteklemez.

PyPy CPython ile uyumlu, RPython (Kısıtlı Python) dili kullanılarak oluşturulmuştur. Bazı kısıtlamalara sahip olsa da CPython'a göre daha hızlıdır. PyPy üretilen ara kodu yorumlayıcı olarak değil JIT Derlemesi (Just in Time Compilation) yaparak çalıştırır.

Jython Java tabanlıdır. Python kodları Java Sanal Makinesinde (JVM) yorumlanacak şekilde derlenir. Java sınıflarını doğrudan kullanabilirler.

- Modul ile Script Farkı?
- import
- pip

Script & Modül Farkı?



package, module, script nedir?

- **script** : import edil**meyen** bir .py uzantılı python dosyası veya bir notebook hücresi
- **module**: import edile**bilen** bir .py uzantılı python dosyası
- **package**: Bir veya daha fazla modül içeren klasör ya da dizin.

Script ?

Script veya program dosyası doğrudan çalıştırılması amaçlanan bir python dosyasıdır.

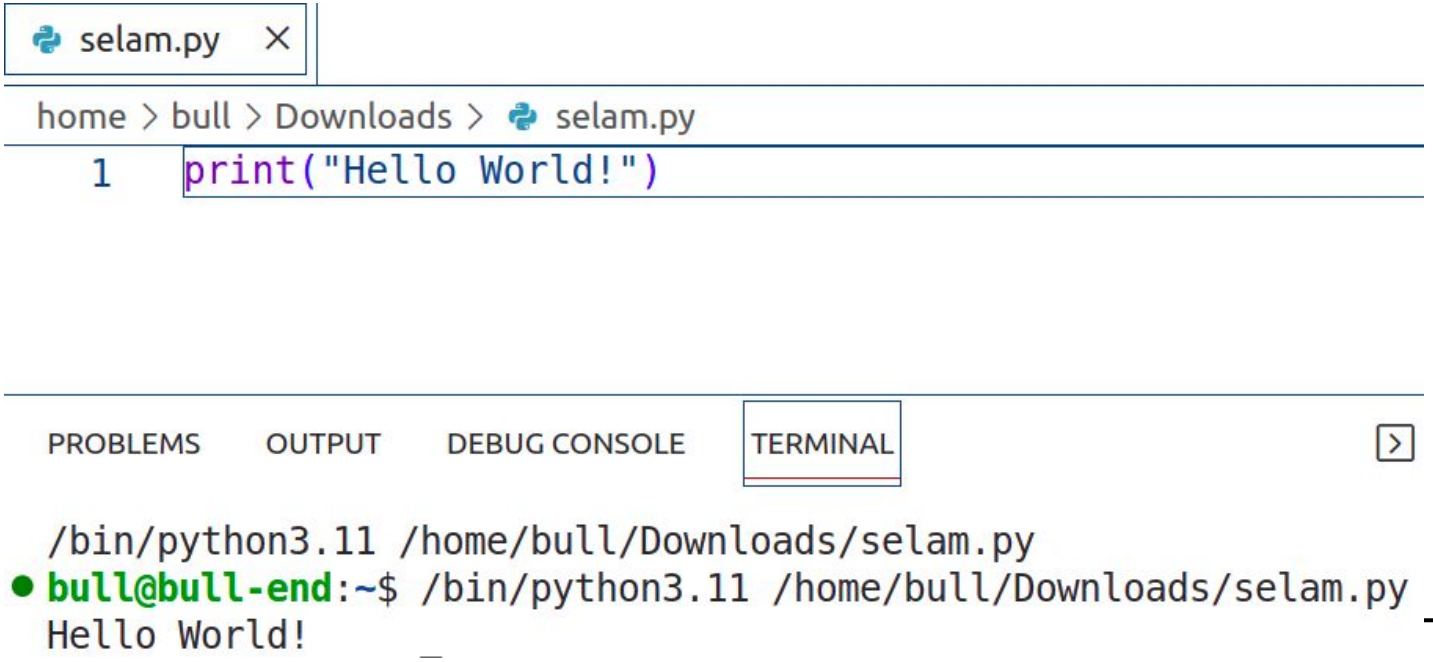
```
print("Hello World!")
```

```
Hello World!
```

Not. Her notebook hücresi bir **script** gibi çalışır.

Script ?

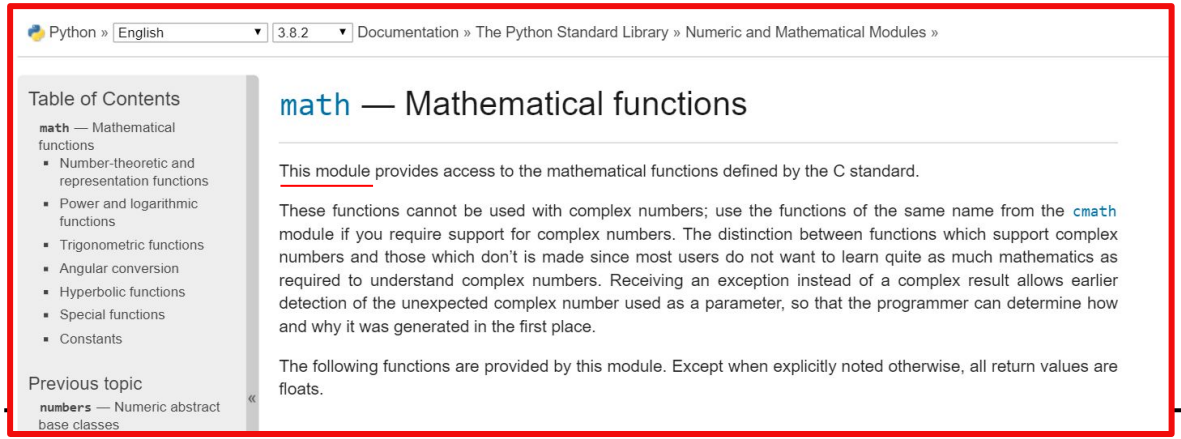
Ama istenirse bir python dosyası (“selam.py” gibi) halinde saklanıp, doğrudan çalıştırılabilir;



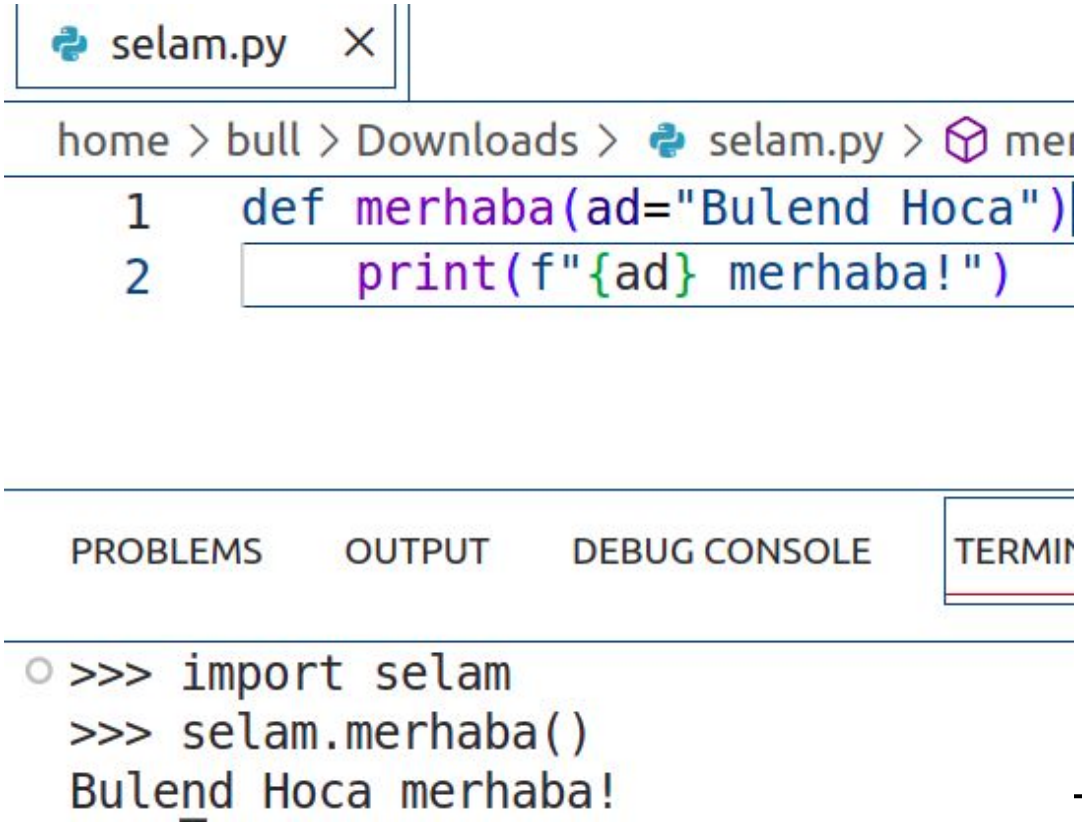
Modul (Module) ?

Modül, diğer .py dosyaları tarafından import edilmesi (içe aktarılması) amaçlanan bir **.py dosyası**dır.

Modüller, belli bir amacı yerine getirmesi amaçlanan fonksiyonları/metotları veya değişkenleri içermelidir.



Modul (Module) ?



Q : Bir modul dosyasını yüklemek için hangi deyim kullanılır?



import ?

```
import modül_adı
```

```
import math
```




```
print(dir(math))
```

✓ 0.3s Python

```
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan',
'atan2', 'atanh', 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp',
'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose',
'isfinite', 'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan',
'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau',
'trunc', 'ulp']
```

import ?

```
import math as m
# math artık m takma adını kullanacak
m.sqrt(9)   # 3.0
# modul_adi.fonksiyon_adi()
```

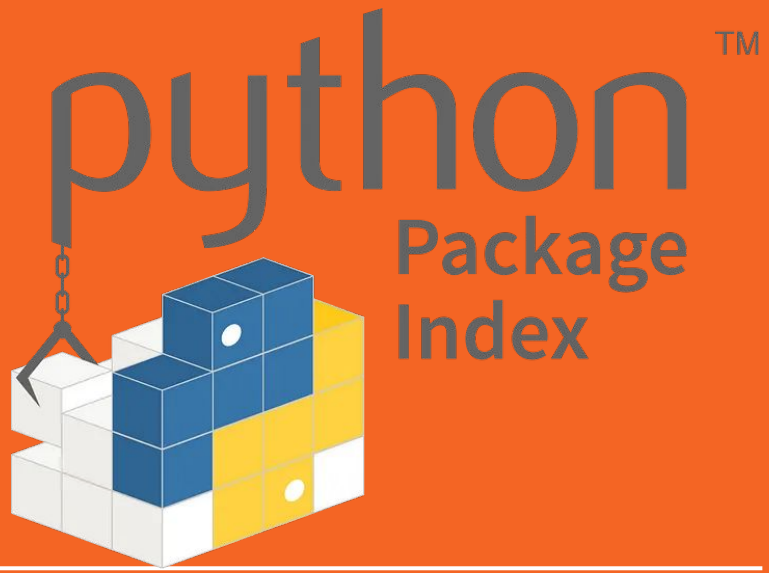
import ?

```
from math import sqrt as kk


# from modul_adi import fonksiyon_adi as takma_ad
kk(9)   # 3.0
```

PyPI (Python Package Index)

- **PyPI**, Python paketlerinin yayınlanması, dağıtılması ve kurulumunu kolaylaştıran bir depodur. Adresi;
- **https://pypi.org/**



– pip ile paket ve modülleri yükleme -

- `pip (Python Installer Packages);` Python'ın **standart paket yönetim** sistemidir.
- **pip**, Python standart kütüphanesinde olmayan ek modülleri/paketleri yönetmek (kurmak/kaldırmak) için kullanılan bir program.
- `pip install numpy`

pip ?

```
pip --version
```

```
!pip --version # colab/jupyter notebook
```

✓ 0.5s

```
pip 22.3.1 from /home/bull/.local/lib/python3.10/site-packages/pip (python 3.10)
```

```
pip install my_package
```

```
pip uninstall my_package
```

pip ?

- Bir paket hakkında özet bilgi almak için;

```
1 pip show my_package
```

- Kurulu paketlerin bir listesini almak için;

```
1 pip list
```

veya

```
!pip freeze
```

Q : pip ile import deyimleri arasındaki fark nedir?



import ile pip farkı

- `import` → Python kodunda bir modülü **içe aktarmak** ve kullanmak için kullanılır.
- `pip` → Python paketlerini **kurmak/yüklemek** için kullanılan paket yöneticisidir.

Örneğin:

```
pip install requests
```

ile `requests` paketini kurarsın. Ardından Python dosyasında:

```
import requests
```

ile paketi programa aktarıp kullanırsın.

Kısaca:
`pip` → **Kur** 📦
`import` → **Kullan** 🐍

Kendi paketimi oluşturma

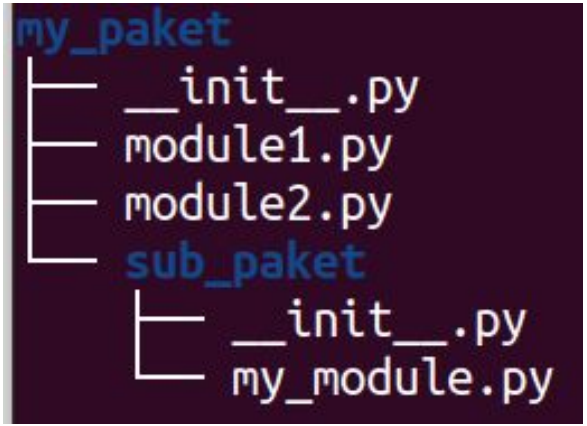
Kendi paket ve alt paket oluşturma

- Bir python paketi içerisinde başka paketlerin ve modüllerin (.py uzantılı dosya) olduğu bir klasördür.

- Her python paketinin içerisinde mutlaka bir tane **__init.py__** isimli bir modülün olması gerekir.

- __init.py__ ; Normal klasörü bir Python paketine dönüştürür.

- Bir paket altındaki modüllere erişim için '.' nokta simgesi kullanılır;
  from <my_paket> import <module1>



from paket.modul import *




```
>>> import pkg.sub_pkg1.mod1
>>> pkg.sub_pkg1.mod1.foo()
foo()

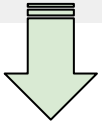
>>> from pkg.sub_pkg1 import mod2
>>> mod2.bar()
bar()

>>> from pkg.sub_pkg2.mod3 import baz
>>> baz()
baz()
```

“Merhaba Dünya”

➔ Python dilinde bir **string (text) mesajı yazdırmak** çok kolaydır.

```
print("Hello World!")
```



```
Hello World!
```

➔ Bu metin(text) mesajı bir **değişkende saklayıp** sonra da **çıktı** olarak yazdırmak da mümkündür:

```
myString = "Hello World!"

print(myString)
```

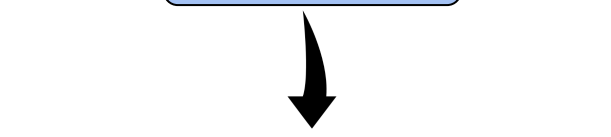


```
Hello World!
```

Tırnak İşaretleri (Quotes)

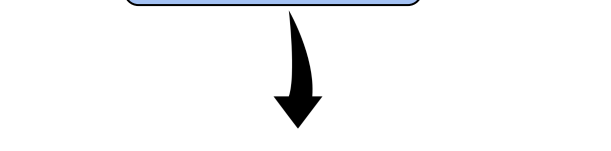
Tırnak içerisine alınan her şey bir string sabittir, mesajdır.

Tek tırnak



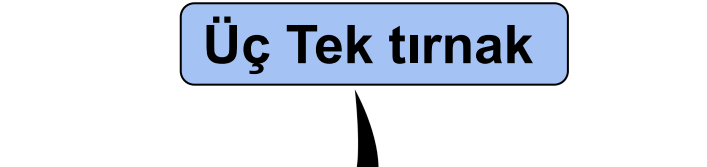
```
print('Hello World!')
```

Çift tırnak



```
print("Hello World!")
```

Üç Tek tırnak



```
print('''Hello World!''')
```

Üç Çift tırnak



```
print("""Hello World!""")
```

Hangi tırnak ile başlandı ise öyle kapatılmalıdır.





```
print('Hello World!')
print("Depar Akademi")
print("""I love you,
    PYTHON""")
print('''Ali, "gel" dedi.''')
```

```
print('Hello World!")
print("Depar Akademi')
print("""I love you,
    PYTHON''')
print("""Ali, "gel" dedi.''')
```

Q : Arasındaki fark nedir?



```
print('3.14159')
print(3.14159)
```

Q : Arasındaki fark nedir?



str

```
print('3.14159')
print(3.14159)
```

float

```
3.14159
3.14159
```

Aynı çıktıyı versek de tipimiz (**type**) farklı!!!

type(): Verinin tipi hakkında bilgi verir.

```
1  print(type("3.14")) # veri tipine barakak farkı anlayabiliri
2  print(type(3.14))
```

✓ 0.5s

```
<class 'str'>
<class 'float'>
```

Yardım Alma: help()

Bir Python komutu hakkında bilgi almak için **`help()`** fonksiyonu kullanılabilir;

Örneğin print() hakkında yardım almak için **`help(print)`** komutu kullanılabilir.

```
1  help(print)
```

✓ 0.2s

```
Help on built-in function print in module builtins:

print(*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.

    sep
      string inserted between values, default a space.
    end
      string appended after the last value, default a newline.
    file
      a file-like object (stream); defaults to the current sys.stdout.
    flush
      whether to forcibly flush the stream.
```

print() fonksiyonunun parametreleri



sep (separator -ayraç): Değerler arasına konacak işaret/simge belirtilir. Default(varsayılan) değer **boşluk**tur.

`print('p','y','t','h','o','n',sep=',')`	**sep (ayraç)** fonksiyonu ekrana basılacak her değerin arasına ',' ile belirtilen karakteri kor. Ekran Çıktısı: **p,y,t,h,o,n**
`print('p','y','t','h','o','n',sep='-')`	Ekran Çıktısı: **p-y-t-h-o-n**

end : En son değerden sonra eklenecek işaret/simge belirtilir. Default(varsayılan) değer olarak bir alt satıra (**newline)** geçilir.

`print("Ad..:",end=' ')` `ad=input()`	**Ad..: Ali** gibi mesaj yanına bir isim girilmesini bekler.
`print("python",end='?')`	Mesaj sonuna '?' karakterini ekler. Ekran çıktısı: **python?**

Q : İki programın çıktısı da aynı mıdır?



```
print("müdür", "müdür", "müdür", sep=" ", end="?")
```

```
print("müdür "*3, end="?")
```

Q : Çıktısı nedir?



```
print("müdür", "müdür", "müdür", sep=" ", end="?")
```

```
müdür müdür müdür?
```

```
print("müdür "*3, end="?")
```

```
müdür müdür müdür ?
```

Çıkış Karakterleri (Escape Characters)

Escape Characters

String ifadelerin çıktı formatını düzenlemede kullanılır.

ters eğik çizgi (\) ile başlar ve ardından temsil etmek istediğiniz karakter veya komut gelir.

```
1  print("Ali \n gel")
```

✓ 0.2s

```
Ali
 gel
```

Escape Sequence	Meaning
`\<newline>`	Backslash and newline ignored
`\\`	Backslash (\)
`\'`	Single quote (')
`\"`	Double quote (")
`\a`	ASCII Bell (BEL)
`\b`	ASCII Backspace (BS)
`\f`	ASCII Formfeed (FF)
`\n`	ASCII Linefeed (LF)
`\r`	ASCII Carriage Return (CR)
`\t`	ASCII Horizontal Tab (TAB)
`\v`	ASCII Vertical Tab (VT)
`\ooo`	Character with octal value *ooo*
`\xhh`	Character with hex value *hh*

Escape Characters

Çıktı formatını düzenleme de '**print()**' ile kullanılabilecek karakterler;

\'	Single quote
\"	Double quote
\\	backslash
\n	New line
\r	Carriage Return
\t	Horizontal tab
\b	Backspace

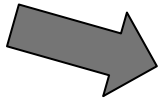
```
>>> print('C:\\Program Files')
C:\Program Files
```

Benzer şekilde, `\'` ve `\"` dizileri sırasıyla tek ve çift tırnak karakterlerini ifade eder:

```
>>> print('\' and \"')
' and "
```

Escape Characters

Çıktı formatını düzenleme de '**print()**' ile kullanılabilecek karakterler;



\N{name}	Unicode Character Database named Lookup
\uxxxxxxxx	Unicode Character with 16-bit hex value XXXX
\Uxxxxxxxx	Unicode Character with 32-bit hex value XXXXXXXX
\ooo	Character with octal value OOO
\xhh	Character with hex value HH

```
1  x = "\N{dog}"
2  print(x)
```

✓ 0.5s

🐕

```
>>> '\N{LATIN CAPITAL LETTER P}'
'P'
>>> '\N{SNAKE}'
'🐍'
```

Escape Characters

Bu dizi, sekizlik (8 tabanlı) sistemde *ooo* değerine sahip bir *karakteri* ifade eder :`\ooo`

```
>>> '\120'
'P'
```

Bu dizi, onaltılık (16 tabanında) değeri *hh* olan bir *karakteri* ifade eder :`\xhh`

```
>>> '\x50'
'P'
```

'r' veya 'R' öneki bulunmadığı sürece, Escape (Çıktı formatı) karakterleri C dili kurallarına göre yorumlanır.

Q : Çıktısı nedir?



```
print('Ali gel\t\'dedi ben olmaz\'\tdedim.')
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Q : Çıktısı nedir?



```
print('Ali gel\t\'dedi ben olmaz\'\tdedim.')
```

```
Ali gel'dedi ben olmaz'  dedim
```

Q : Çıktıları hangi hayvana benzer?




```
print(" /\_/\\\n( o.o )\n > ^ <\tCat")
```

```
print(" ,,\n(o)<\tMax\n ``")
```

https://www.asciiart.eu/

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Task : Sıra Sende:

Tek bir print() deyimini kullanarak aşağıdaki karakteri ekrana yazdırabilir misiniz?



```
\     /\
 )   ( ')
(   /   )
 \(__)|
```

miyav


```
\     /\
 )   ( ')
(   /   )
 \(__)|
```

miyav

https://www.asciiart.eu/

Q : Sıra Sende:

Tek bir print() deyimini kullanarak aşağıdaki karakteri ekrana yazdırabilir misiniz?



```
print("\t\     /\\\n\t )   ( ')\n\t(   /   )\nmiyav\t \(__)|")
```

```
\     /\
 )   ( ')
(   /   )
 \(__)|
```

miyav

https://www.asciiart.eu/

ASCII Kodu Elde Etme

Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char
48	30	110000	60	0	96	60	1100000	140	`
49	31	110001	61	1	97	61	1100001	141	a
50	32	110010	62	2	98	62	1100010	142	b
51	33	110011	63	3	99	63	1100011	143	c
52	34	110100	64	4	100	64	1100100	144	d
53	35	110101	65	5	101	65	1100101	145	e
54	36	110110	66	6	102	66	1100110	146	f
55	37	110111	67	7	103	67	1100111	147	g
56	38	111000	70	8	104	68	1101000	150	h
57	39	111001	71	9	105	69	1101001	151	i
58	3A	111010	72	:	106	6A	1101010	152	j
59	3B	111011	73	;	107	6B	1101011	153	k
60	3C	111100	74	<	108	6C	1101100	154	l
61	3D	111101	75	=	109	6D	1101101	155	m
62	3E	111110	76	>	110	6E	1101110	156	n
63	3F	111111	77	?	111	6F	1101111	157	o
64	40	1000000	100	@	112	70	1110000	160	p
65	41	1000001	101	A	113	71	1110001	161	q
66	42	1000010	102	B	114	72	1110010	162	r
67	43	1000011	103	C	115	73	1110011	163	s
68	44	1000100	104	D	116	74	1110100	164	t
69	45	1000101	105	E	117	75	1110101	165	u
70	46	1000110	106	F	118	76	1110110	166	v
71	47	1000111	107	G	119	77	1110111	167	w
72	48	1001000	110	H	120	78	1111000	170	x
73	49	1001001	111	I	121	79	1111001	171	y
74	4A	1001010	112	J	122	7A	1111010	172	z

chr() :

```
chr(97)    # 'a'
```

ord() :

```
ord("a")   # 97
```

https://docs.python.org/3/library/functions.html#chr

Sayılardaki ön takılar



octal

```
print(0o21)
print(0x11)
print(0b10001)
```

hexadecimal

binary

https://www.asciiart.eu/

Sayı Sistemleri ve ön takılar

Ön Takı (Prefix)	Sistem	Taban	Sayılar
Yok	Decimal (Onlu)	10	0,1,2,3,4,5,6,7,8,9
0b (zero + lowercase letter 'b') **0B** (zero + uppercase letter 'B')	Binary(İkili)	2	0,1
0o (zero + lowercase letter 'o') **0O** (zero + uppercase letter 'O')	Octal(Sekizli)	8	0,1,2,3,4,5,6,7
0x (zero + lowercase letter 'x') **0X** (zero + uppercase letter 'X')	Hexadecimal(Onaltılı)	16	0,1,2,3,4,5,6,7,8,9,**A,B,C,D,E,F**

Sayılardaki Ön Takılar

Sayı Sistemlerinin Gösteriminde Kullanılan Ön Takılar Python dili C tabanlı bir dil olduğu için sayı sistemlerinin gösteriminde C dili ön takıları kullanılır. Bunlar;

- # Onlu sayı sisteminde öntakı yoktur.
- İkili (Binary) sayı sisteminde, **0b** ön takısı,
- Sekizli (Octal) sayı sisteminde, **0o** (Sıfır ve o) ön takısı,
- Onaltılı (Hexadecimal) sayı sisteminde ise **0x** ön takısı kullanılır.

```python
# Buna göre onluk sistemde verilen 17 sayısı, aşağıdaki gibi dört farklı şekilde ifade edilebilir:
onlu = 17
ikili = 0b10001
sekizli = 0o21
onaltili = 0x11
print(onlu)
print(ikili)
print(sekizli)
print(onaltili) # hepsi aynı çıktıyı verir
```

✓ 0.2s Python

```
17
17
```

input()
ile Veri Girişi

input()

Python'da klavyeden veri girişi için input() kullanılır.

```
ad = input('Adınız?')
# Bu kullanımda klavyeden girilen veri, string(sözel) ad değişkeninde tutulur.
# Varsayılan tip değeri string'dir.
```

```
nu = int(input('Numaranız?'))
# Bu kullanımda klavyeden girilen veri, int (tamsayı) tipindeki nu değişkeninde tutulur.
```

```
b = float(input('Maasiniz?'))
# Bu kullanımda klavyeden girilen veri, float (kesirli sayı) tipindeki b değişkeninde tutulur.
```

```
c = eval(input("İfade..: "))
# Bu kullanımda klavyeden girilen verinin tipi otomatik algılanır ve c değişkeninde tutulur.
```

Q : ‘Ali’ girilirse çıktı ne olur?



```
word = input("Word: ") # "Ali"
sep = '/'
tekrar = 3
print((word + sep) * tekrar + '\b', end = "?")
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Q : ‘Ali’ girilirse çıktı ne olur?



```
word = input("Word: ") # "Ali"
sep = '/'
tekrar = 3
print((word + sep) * tekrar + '\b', end = "?")
```

```
Word: Ali
Ali/Ali/Ali?
```

Task : Girilen bir ismi 5 kez alt alta nasıl yazdırabiliriz?



```
Ad: Bulend
Bulend
Bulend
Bulend
Bulend
Bulend
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Q : Girilen bir ismi 5 kez alt alta nasıl yazdırabiliriz?



```
print((input("Ad: ") + "\n" )*5)
```

```
Ad: Bulend
Bulend
Bulend
Bulend
Bulend
Bulend
```

PEP 8 Kuralları
Temiz Kod

PEP (Python Enhancement Proposal)

PEP, konularına göre indekslenmiş, Python Geliştirme Önerileri dir.

	PEP	Title	Authors
PA	1	PEP Purpose and Guidelines	Warsaw, Hylton, Goodger, Coghlan
PA	2	Procedure for Adding New Modules	Cannon, Faassen
PA	4	Deprecation of Standard Modules	Cannon, von Löwis
PA	5	Guidelines for Language Evolution	Prescod
PA	6	Bug Fix Releases	Aahz, Baxter
PA	7	Style Guide for C Code	GvR, Warsaw
PA	8	Style Guide for Python Code	GvR, Warsaw, Coghlan
PA	10	Voting Guidelines	Warsaw
PA	11	CPython platform support	von Löwis, Cannon
PA	12	Sample reStructuredText PEP Template	Goodger, Warsaw, Cannon
PA	387	Backwards Compatibility Policy	Peterson
PA	581	Using GitHub Issues for CPython	Mariatta
PA	609	Python Packaging Authority (PyPA) Governance	Ingram, Gedam, Harihareswara
PA	676	PEP Infrastructure Process	Turner

https://peps.python.org/

PEP- 8 (Style Guide for Python Code)

PEP-8, yazılan kodun **tutarlılığı (***consistency***)** ile ilgilenir.

- Guido' ya göre yazılımcının vakti, kod yazmaktan daha çok okumakla geçer,
- PEP 20 'de söylendiği gibi "**Okunabilirlik önemli**dir -*Readability counts*".
- PEP-8 de belirtilen kod **yazım kuralları önemli**dir. Bu kurallara uymak yazdığın kodun tutarlılığını sağlar.
- Fakat bir proje içerisindeki **kod tutarlılığı** daha **önemli**dir.
- Bir modül veya fonksiyon içerisindeki **tutarlılık** daha da **önemli**dir.

https://peps.python.org/pep-0008/

Bazı önemli PEP8 Kuralları

```
print("Faydalı bir insan olacağım")
```

Bir kod satırındaki maksimum karakter sayısı: 79 ama

```
"""This is the example docstrings.
'Comments or docstrings' are not printed on the screen.
"""

#being a good person …
```

Yorum satırları ve docstring belgeleri için; **72 karakter**

https://peps.python.org/pep-0008/

Bazı önemli PEP8 Kuralları

- Tab mı Boşluk mu?





Girinti(*indentation*) için **4 boşluk(space)** bırakınız.

https://peps.python.org/pep-0008/

Bazı önemli PEP8 Kuralları

- **Tab mı Boşluk mu?** Girinti bırakmak için spaces (boşluklar) tercih edilmeli.

```
# Wrong:

# Arguments on first line forbidden when not using vertical align
foo = long_function_name(var_one, var_two,
    var_three, var_four)

# Further indentation required as indentation is not distinguisha
def long_function_name(
    var_one, var_two, var_three,
    var_four):
    print(var_one)
```

```
# Correct:

# Aligned with opening delimiter.
foo = long_function_name(var_one, var_two,
                         var_three, var_four)

# Add 4 spaces (an extra level of indentation) to distinguish arguments from the rest.
def long_function_name(
        var_one, var_two, var_three,
        var_four):
    print(var_one)
```

https://peps.python.org/pep-0008/

Bazı önemli PEP8 Kuralları

- İçe aktarma işlemleri genellikle ayrı satırlarda yapılmalıdır:

```
# Correct:
import os
import sys
```

-
```
# Wrong:
import sys, os
```

- Bunu söylemekte sakınca yok:

```
# Correct:
```

-
```
from subprocess import Popen, PIPE
```

İçe aktarma işlemleri her zaman kod satırlarının **en üstüne,** modül yorumlarından ve docstring metinlerinden hemen sonra yazılır.

---

https://peps.python.org/pep-0008/

Bazı önemli PEP8 Kuralları

- **Gereksiz boşluklara gerek yok!**





`# Correct:` `foo = (0,)`	`# Wrong:` `bar = (0, )`
`spam(ham[1], {eggs: 2})`	`spam( ham[ 1 ], { eggs: 2 } )`
`spam(1)`	`spam (1)`
`x = 1` `y = 2`	`x = 1` `y = 2`
`dct['key'] = list[index]`	`dct ['key'] = list [index]`
`# Immediately before a comma, semicolon, or colon` `if x == 4: print(x, y); x, y = y, x`	`if x == 4 : print(x , y) ; x , y = y , x`

https://peps.python.org/pep-0008/

Ders nasıl gidiyor?



Sıkıcı



Eh işte!



İyi, akıcı

Değişkenler (Variables)
ve
PEP-8 Önerileri

Sabit Tanımlaması

Programın çalışması boyunca değeri değişmeyen ifadelere **sabit(constant/**literal) denir.

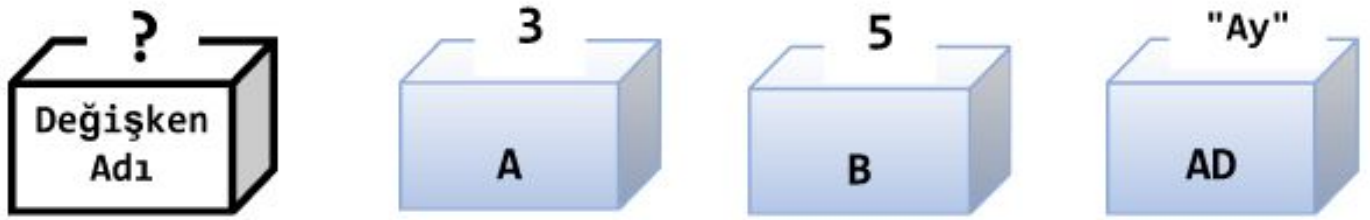
Bir PEP-8 önerisi olarak sabitler büyük harfle yazılır;

```
FILE = 'setup.cfg'
```

```
PI = 3.14
```

Değişkenler (Variables)

- Değişkenler içerisinde veri (değer) sakladığımız küçük **bellek(**hafıza) **alanları**dır. **Değişkenleri** üzerine etiket yapıştırılan **saklama kapları**na benzetebiliriz. Programın çalışması boyunca bu saklama kaplarındaki veriler(değerler) değişebilir. Bu nedenle bu saklama kaplarına '**değişken**' adı verilmiştir.



Değişkenler (Variables)



Matematiksel ya da sözel bir değeri bir değişkene aktarmak/atamak için **'='** **eşittir** operatörü kullanılır;

```
sayi = 3
int_num = 5
ad = "Ali"
```

Değişkenler (Variables)

👇**Q: Aşağıdaki programın çıktısı nedir?**

```
>>> first_number = 400
>>> second_number = first_number
>>> print(second_number)
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Değişkenlere Değer Atama

```
1  first_number = 400
2  second_number = first_number
3  print(second_number)
4
```

✓ 0.0s

```
400
```



Değişkenlere Değer Atama

Q: 👇Aşağıdaki gibi birden fazla değişkene tek satırda değer atanabilir mi?

```
a, b, c = 5, 3.2, "Hello"
print(b)
print(c)
print(a)
```



Students choose an option

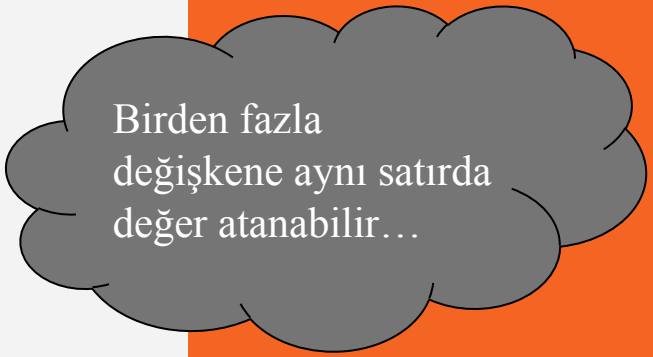
Pear Deck Interactive Slide
Do not remove this bar

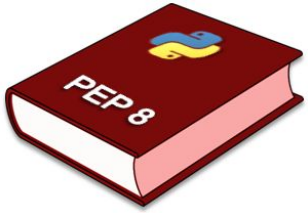
Değişkenlere Değer Atama

```
1  a, b, c = 5, 3.2, "Hello"
2
3  print(a)
4  print(b)
5  print(c)
6
```

✓ 0.0s

```
5
3.2
Hello
```



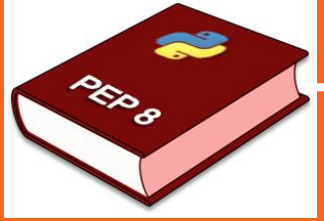


Değişken Tanımlama Kuralları

- **İsimler harf veya '_'** ile başlayabilir. **Ama rakam ve diğer özel karakter ile başlayamaz**

  55a, **+55** yanlış iken **a55, _55** doğrudur.
  ad soyad yanlış iken **ad_soyad** doğrudur.

- Python **keyword** (anahtar kelimeleri) değişken ismi olarak **kullanılamaz.**



keyword listesi

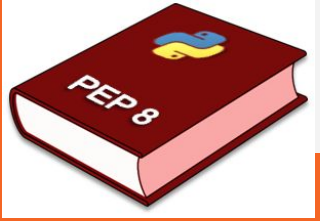
Güncel keyword listesini aşağıdaki kod satırları ile öğrenebiliriz;

```
import keyword
print(keyword.kwlist)
```

keyword listesi

```
False      await      else       import     pass
None       break      except     in         raise
True       class      finally    is         return
and        continue   for        lambda     try
as         def        from       nonlocal   while
assert     del        global     not        with
async      elif       if         or         yield
```

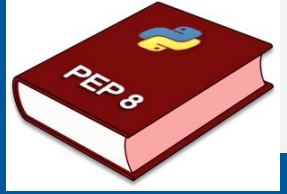
not yanlış iken **Not** doğrudur.
None yanlış iken **none** doğrudur.



Değişken Tanımlama Kuralları

- 2me 👉 😔
- Big_boss 👉 😔
- not 👉 😔
- big-boss 👉 😔
- 1453Fatih 👉 😔
- first! 👉 😔
- $price 👉 😔
- xy#@!v 👉 😔

data4me 👉 😀
big_boss 👉 😀
Not 👉 😀
fatih1453 👉 😀
last_name 👉 😀
_first 👉 😀
price_dolar 👉 😀
_1_2_3 👉 😀

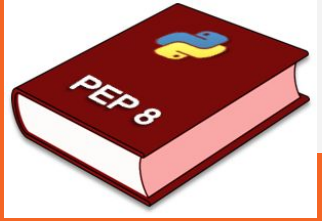


Değişken Tanımlama Kuralları

- Kabul edilse de Türkçe karakterler(ş,ü,ğ,ç,ö,ı) kullanılmamalı,

- Küçük harf (lowercase) tercih edilmeli,
  NO no
- Birden fazla kelimeden isimler ‘_’ ile ayrıştırılmalı,
  weekly_pay
  ad_soyad
- keyword listesinde olmayan Python komutları (print, list, type gibi) da isimlendirme de kullanılmamalı,

- Alfabenin karşıklığa neden olabilecek harflerini (o, 0 l, I) kullanmayın,



Değişken Tanımlama Kuralları

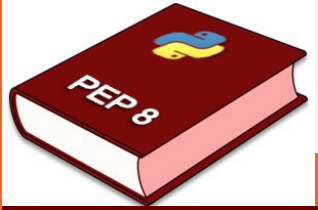
- **İsim karışıklığına sebep olacak karakterler kullanmayın.**
  Örneğin;

```
a = b * c
```

yerine

```
weekly_pay = hours_worked * hourly_pay_rate
```

tercih edilebilir.



Değişken Tanımlama Kuralları

- **İyi İsimlendirme 😀**

```
weekly_pay

big_data

numara

asal_sayilar

avg_income_feb
```

Kötü İsimlendirme 😔

```
w_p

b_d

n

asal_sayilar_listesi

# Average income of February
average_income_of_february
```

Değişkenlerin Yazdırılması

Bir değişkenin içeriğini ekrana yazdırmanın 5 farklı yolu;👇

```
A = 3
print('A =',A)
print('A = %d' %A)                    #C tarzı kullanım
print("A = {0}".format(A))
print(f"A = {A}")
print('A = ' + str( A ))              #Java tarzı kullanım
```

İçi boş bir değişken nasıl tanımlanır?

```
# Eğer bir değişkene hangi değeri atayacağımızı baştan
# bilmiyorsak, ona ne atayabiliriz?
# Örneğin eğitmenin yaşı : ?
```

```
thomas = 33
nihal = 28
bulent = ?
faruk = 46
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

İçi boş bir değişken nasıl tanımlanır?

```
thomas = 33
nihal = 28
bulent = None
faruk = 46
```

SWAP(Yer değiştirme)



Q: 👇

- İki değişkenin **ali = ‘A’; veli = ‘V’** içeriğini yer değiştiren uygulama nasıl kodlanabilir?

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Comments (Yorum Satırları)

1- Tek satırlı yorumlar için;

```
# Bu bir tek satırlı yorumdur.
```

2- Satır içi yorumlar için;

```
print('hello')  # Bu bir satır içi yorumdur.
```



Comments (Yorum Satırları)

3- Çok satırlı yorumlar için;

```
print('hello')
# İlk yorum satırı,
# İkinci yorum satırı,
# üçüncü yorum satırı
```

Yukarıdaki programın çıktısı nedir?



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Comments (Yorum Satırları)

3- Çok satırlı yorumlar için;

```
print('hello')
# İlk yorum satırı,
# İkinci yorum satırı,
# üçüncü yorum satırı
```

```
hello
```

Comments (Yorum Satırları)

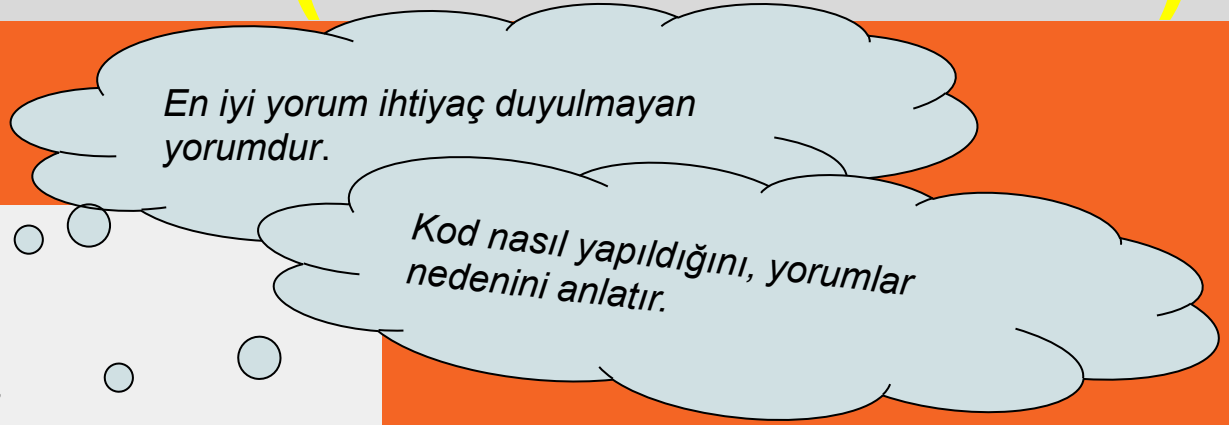
4- Blok yorumlar;

```
print('hello')
##################################
#                                #
# This is the comment body.      #
# Variation Two.                 #
#                                #
##################################
```

Comments (Yorum Satırları)

Yorumlar;

- **Verimli**
- **Gerektikçe**
- **Güncel**



- A bad comment is worse than no comment at all.

https://stackoverflow.blog/2021/12/23/best-practices-for-writing-code-comments/

Q: Yorumlar kimin için yazılır ya da yazılmalıdır?



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Docstrings (Documentation strings)

(Dokümantasyon)

Docstring (Documentation strings)

Docstring Nedir?

Docstring, bir modül/fonksiyon/sınıf veya metodun işlevini **açıklamada/tanımlamada** ilk ifade olarak ortaya çıkan bir **string sabittir.**

Bir docstring ifadesine, o nesnenin **__doc__** özelliği ile erişebiliriz.

Docstring (Documentation strings)

Docstring Nasıl Tanımlanır?

- Docstring ifadeleri - yorumların aksine- yorumladıkları/açıkladıkları modül veya fonksiyonun bir özelliği(attribute) olarak depolanırlar.
- Yani bir docstring metnine erişilebilinir.

- Docstring ifadelerinde 3 tek ya da çift tırnak kullanılır;

```
"""...docstring metni..."""
```

Docstring (Documentation strings)

Docstring Nasıl Tanımlanır?

- Docstring ifadeleri - yorumların aksine- yorumladıkları/açıkladıkları modül veya fonksiyonun bir özelliği(attribute) olarak depolanırlar.
- Yani bir docstring metnine erişilebilinir.

- Docstring ifadelerinde 3 tek ya da çift tırnak kullanılır;

```
"""...docstring metni..."""
```

Docstring (Documentation strings)

Docstring Nasıl Tanımlanır?

```
def fonksiyon_adi():
    """
    Ben bir docstring metniyim.
    Fonksiyonun ne iş yaptığını açıklarım.
    Ve genellikle en başa yazılırım.
    Üç ''' tırnak """ kullanırım.
    """
    pass
    ….
….
```

Fonksiyonun docstring'i

Fonksiyonun kod bloğu

Docstring (Documentation strings)

Bir fonksiyonun docstring metnine nasıl erişilir?

fonksiyon_adi .__doc__
özelliği ile
veya **help**(fonksiyon_adi)
fonksiyonu ile bir docstring metni okunabilir.

```
def fonksiyon_adi():
    """
    Ben bir docstring metniyim.
    Fonksiyonun ne iş yaptığını açıklarım.
    Ve genellikle en başa yazılırım.
    Üç ''' tırnak """ kullanırım.
    """
    pass

#ana program
print(fonksiyon_adi.__doc__)
```

Çıktı

```
Ben bir docstring metniyim.
Fonksiyonun ne iş yaptığını açıklarım.
Ve genellikle en başa yazılırım.
Üç ''' tırnak """ kullanırım.
```

Docstring (Documentation strings)

1. Örneğin **print() f**onksiyonun **docstring** metnine aşağıdaki gibi erişebiliriz;

```
print(print.__doc__)
```



```
print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

Prints the values to a stream, or to sys.stdout by default.
Optional keyword arguments:
file:  a file-like object (stream); defaults to the current sys.stdout.
sep:   string inserted between values, default a space.
end:   string appended after the last value, default a newline.
flush: whether to forcibly flush the stream.
```

Docstring (Documentation strings)

2. Örneğin **print() f**onksiyonun **docstring** metnine aşağıdaki gibi erişebiliriz;

```
help(print)
```



```
print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

Prints the values to a stream, or to sys.stdout by default.
Optional keyword arguments:
file:  a file-like object (stream); defaults to the current sys.stdout.
sep:   string inserted between values, default a space.
end:   string appended after the last value, default a newline.
flush: whether to forcibly flush the stream.
```

Task: Sizde aşağıdaki fonksiyonların docstring metinlerine bakarak işlevlerini açıklamaya çalışınız?



```
input()    : print(input.__doc__)
sum()      : ...
type()     : …
hex()      : …
bin()      : …
```

Docstring kullanımı

- **Docstring'ler** ayrıca **test yazılımlarında** kullanılabilir. Bunun için **doctest** modülünün import edilmesi gerekir.

```
def my_add(a,b):  #test fonksiyonu
    """
    >>> my_add(2,3)
    5
    >>> my_add(3,-6)
    -3
    >>> my_add(2,3)
    6
    """
    return a+b
# ana program
import doctest
doctest.testmod()
```

Çıktısı

```
**********************************************************************
File "__main__", line 8, in __main__.my_add
Failed example:
    my_add(2,3)
Expected:
    6
Got:
    5
**********************************************************************
1 items had failures:
   1 of   3 in __main__.my_add
***Test Failed*** 1 failures.
TestResults(failed=1, attempted=3)
```

Bugünkü dersi anladık mı?





Bugünkü dersin işleyiş hızı nasıldı?



Çok yavaş



İyi



Çok hızlı

Eğitmenin performansından memnun kaldık mı?



